Role-Isolated State Partitions and Asymmetric Verification Gates for Multi-Agent Software Synthesis

Onur Aksoy
Hackonomist Applied Systems Architecture, Ankara, Turkey
dev@onuraksoy.com.tr | ORCID: 0009-0001-4481-1759
Published: 14 August 2026 | Technical Note | Distributed under CC BY 4.0

Abstract

Autonomous multi-agent systems driven by Large Language Models (LLMs) are increasingly applied to complex software engineering tasks. However, current implementations relying on monolithic conversational contexts or unconstrained peer-to-peer topologies exhibit three recurring failure modes: context pollution, concurrent filesystem race conditions, and self-grading verification bias. When multiple agents modify an active codebase simultaneously, mutual read-write contention over repository index files and shared type definitions produces irrecoverable divergence and phantom test passes. This technical note presents a deterministic orchestration architecture grounded in role-isolated state partitions and asymmetric verification gates. By partitioning the codebase filesystem into mathematically disjoint write sets assigned to specialized roles, concurrent write contention is reduced to zero. In addition, an independent dual-stage verification layer separating build execution from semantic audit eliminates developer self-evaluation bias. An empirical evaluation across 48 software modification tasks demonstrated that this architecture eliminated write conflicts (18.7% to 0.0%), lowered post-build defect rates from 22.9% to 2.1%, and reduced mean context token consumption by 31.4% compared to monolithic architectures.

Keywords: Multi-Agent Systems, Autonomous Software Synthesis, Concurrency Control, Asymmetric Verification, Formal Partitions, Large Language Models, Static Analysis.

1. Introduction and Problem Statement

Modern software engineering tasks require modifying existing architectures without breaking existing operational invariants. While single-agent LLM systems have demonstrated proficiency on localized programming benchmarks, their reliability declines sharply when tasked with multi-file modifications in mature repositories. Deploying teams of specialized agents has emerged as a natural design paradigm. However, unconstrained multi-agent architectures introduce operational failures that stem directly from how state and verification responsibilities are distributed.

In typical conversational agent topologies, agents operate over a shared working directory and append conversational dialogue to a communal context log. In production codebases, this strategy produces three critical systemic failures:

Context Pollution and Token Saturation: When intermediate debugging outputs, compiler stack traces, and full file contents are injected into a single shared context window, irrelevant tokens dilute the attention mechanism. Agents misplace architectural constraints and overwrite previously stabilized modules as context length expands.

Concurrent Filesystem Race Conditions: When multiple agents execute concurrent file writes, simultaneous modifications to shared interface files and the local version control index produce lock collisions (e.g., .git/index.lock contention) and silent overwrites. Because LLM agents lack internal synchronization primitives, race conditions corrupt the repository state.

Self-Grading Verification Bias: In unconstrained systems, the generator agent that authors a code patch is typically tasked with testing and approving its own solution. This creates a correlated failure pattern: the agent hallucinates that its implementation is correct, comments out failing assertions, or substitutes genuine operational validation with trivial mock stubs.

To resolve these pathologies, this paper formulates a deterministic orchestration model based on disjoint filesystem partitions, asymmetric verification gates, and strict escalation limits.

2. Formal Partitioning Model

We model a software repository as a finite filesystem tree F consisting of all valid file paths. Let R represent the set of specialized agent roles instantiated within the orchestration runtime:

Symbol	Agent Role	Write Scope W(r_i)	Operational Boundary Description
r_data	Backend / Data	src/content/, src/db/, src/api/	Backend services, database migrations, data schemas, API handlers.
r_ui	Frontend / UI	src/components/, src/layouts/	Visual presentation, component styles, layouts, view templates.
r_contracts	Contracts Agent	docs/API.md, shared/types/	Exclusive authority over API contracts and shared data schemas.
r_tester	Tester Agent	tests/, e2e/	Build execution and integration test authoring. Application code immutable.

r_auditor	Strict Auditor	empty_set (Read-Only)	Zero-write security, a11y, schema validation, and mock detection.

2.1. Formal Safety and Disjointness Invariants

The partition model guarantees state consistency through three fundamental invariants:

Invariant 1 (Spatial Disjointness): For any two distinct developer roles r_i, r_j in R with i != j, their write domains are mutually disjoint: W(r_i) intersect W(r_j) = empty_set. Because write sets never overlap, agents can execute tasks within their domain without filesystem lock contention.

Invariant 2 (Monotonic Contract Mutability): The shared contract domain F_schema = docs/API.md union shared/types/ possesses exactly one writer: W(r_contracts) = F_schema. All other roles maintain read-only access. When a feature requires schema expansion, r_contracts must commit the updated contract before developer agents may consume it.

Invariant 3 (Read-Only Auditor Isolation): W(r_auditor) = empty_set. The auditor is cryptographically and operationally barred from modifying source files, preventing verifier tampering or recursive self-correction cycles.

Invariant 4 (Serialized Commit Authority): Developer agents do not execute version control commit commands directly. Local git commits are scheduled exclusively by the orchestrator upon receiving dual-stage verification sign-off, completely preventing .git/index.lock contention.

2.2. Isolated Context Architecture

To eliminate cross-agent context pollution, the global chat log is eliminated. Each agent r_i reads and updates an isolated markdown ledger: context/r_i.md. When an agent completes a task, it appends a structured summary including modified file paths, verification command outputs, and explicit assumptions. The root orchestrator maintains high-level phase status in a central CONTEXT.md, ensuring that developer contexts remain compact and focused on domain logic.

3. Asymmetric Verification and Escalation Ceiling

Automated code synthesis requires verification mechanisms that do not inherit the assumptions or hallucinations of the generator. We implement this via an asymmetric two-stage gate coupled with a strict escalation ceiling.

3.1. Dual-Stage Asymmetric Verification Gate

A patch P produced by a developer agent must pass through two independent evaluation filters before reaching the orchestrator for commit:

Verification Gate	Agent Assigned	Validation Modality	Rejection Criteria
Gate 1: Execution	Tester Agent	Containerized build, type check, unit tests, link check	Exit code != 0; unhandled compiler error; broken internal link; te
Gate 2: Semantic	Strict Auditor	AST analysis, mock/TODO scanning, schema entry validation	Placeholder comments (TODO/FIXME); dummy mock stubs; m

Model Reasoning Asymmetry: To guarantee thorough auditing, the orchestrator enforces an asymmetry constraint on model capacity: Capacity(M_auditor) >= Capacity(M_developer). While code generation can be distributed to high-throughput models, the auditing role is pinned to high-capacity reasoning models. This prevents developer hallucinations from confusing the verifier.

3.2. Bounded Escalation Ceiling (N = 3)

Unconstrained agent loops frequently succumb to token runaway: when faced with a subtle compiler or architectural error, an agent repeatedly applies minor variations of the same faulty patch, consuming substantial token budgets without converging. We formalize this bound as a strict escalation ceiling:

Let Rejections(task) denote the cumulative rejection count from either verification gate. If Rejections(task) >= 3, the orchestrator aborts the task immediately and emits a structured Human Escalation Report. The report aggregates: (a) initial specification, (b) diffs from all three attempts, (c) execution logs and compiler diagnostics, and (d) the auditor's rejection rationale. Automated retries cannot resume without human intervention, guaranteeing finite loop termination.

```
Algorithm 1: Deterministic Partitioned Orchestration Loop
Input: Task specification T, Target Role r_dev, Filesystem F, Contexts C
Output: Committed Patch P or Escalation Report E
1: Initialize retry counter: k <- 0
2: while k < 3 do
3:     P <- r_dev.Synthesize(T, C[r_dev]) subject to WriteScope(P) subset of W(r_dev)
4:     if WriteScope(P) not subset of W(r_dev) then Reject('Out-of-scope write'); k <- k + 1;
continue
5:     (exit_code, log) <- r_tester.ExecuteBuildAndTests(F + P)
6:     if exit_code != 0 then Reject('Build/Test failure', log); k <- k + 1; continue
7:     (verdict, audit_notes) <- r_auditor.AuditSemanticIntegrity(P, F + P)
8:     if verdict == APPROVED then
9:         Orchestrator.Commit(P); r_dev.UpdateContext(C[r_dev]); return P
10:    else Reject('Audit rejection', audit_notes); k <- k + 1
11: end while
12: E <- Orchestrator.GenerateEscalationReport(T, history); return E
```

4. Empirical Evaluation

To validate the architecture, we conducted an empirical evaluation across 48 software modification tasks in active multi-language production codebases. The test suite was balanced across three representative technology stacks: (1) 16 tasks in Astro/TypeScript (front-end layout, reactive state, and edge routing), (2) 16 tasks in Rust (system utilities, high-throughput network packet capture, and CLI tools), and (3) 16 tasks in C#/.NET 9 (enterprise transaction APIs and data models). Tasks included schema migrations, API contract revamps, responsive layout adjustments, and database indexing updates.

Each task was evaluated independently under three distinct configurations:

- **Configuration A (Single-Agent Monolithic Context):** A single high-capacity LLM managing code authoring, build execution, and sign-off within a single context.
- **Configuration B (Unconstrained Chat Ring):** Multiple specialized agents communicating in a broadcast conversational topology over a shared workspace without write restrictions.
- **Configuration C (Role-Isolated Partitioned Orchestration):** The proposed model enforcing disjoint filesystem write sets, isolated context ledgers, dual verification gates, and N = 3 escalation ceiling.

Evaluation Metric	Monolithic Context	Unconstrained Chat Ring	Role-Isolated Partitioned
Task Sample Size (Astro / Rust / C#)	48 (16 / 16 / 16)	48 (16 / 16 / 16)	48 (16 / 16 / 16)
Filesystem Write Conflict Rate (%)	0.0%	18.7% (9 / 48)	0.0% (0 / 48)
Uncaught Post-Build Defect Rate (%)	27.1% (13 / 48)	22.9% (11 / 48)	2.1% (1 / 48)
Mean Context Tokens per Task (k)	142.8	218.4	97.9
Token Reduction vs Chat Ring (%)	Base (-34.6%)	+52.9%	-55.2%
Loop Termination Failures (%)	12.5% (6 / 48)	16.7% (8 / 48)	0.0% (0 / 48)
Automated Human Escalations (N=3)	0 (Unbounded loops)	0 (Unbounded loops)	3 (6.25%)
Median Task Completion Time (s)	184	312	146

4.1. Defect Classification and Qualitative Findings

To understand the distribution of defects, all post-build failures were classified into three failure modes: (1) *Mock/Stub Pollution* (inserting dummy data instead of genuine integration), (2) *Contract Drift* (inconsistencies between frontend fetch and backend API routes), and (3) *Lock Contention Failures* (.git/index.lock conflicts and simultaneous file truncations).

Failure Mode Observed	Monolithic Context	Unconstrained Chat Ring	Role-Isolated Partitioned
Mock/Stub Subsitution	7 occurrences (14.6%)	6 occurrences (12.5%)	0 occurrences (0.0%)
Contract & Schema Drift	4 occurrences (8.3%)	5 occurrences (10.4%)	1 occurrence (2.1%)
Filesystem Lock Contention	0 occurrences (0.0%)	9 occurrences (18.7%)	0 occurrences (0.0%)
Unbounded Token Runaway (>10 iters)	6 occurrences (12.5%)	8 occurrences (16.7%)	0 occurrences (0.0%)

Key Empirical Observations:

1. *Complete Suppression of Write Collisions:* In the unconstrained chat ring, 9 of 48 runs failed due to simultaneous edits to shared route files or concurrent git operations. In Configuration C, zero write collisions occurred across all 48 tasks.

2. *Auditor Rigor Against Mock Pollution:* In Configurations A and B, developer agents frequently introduced mock arrays or disabled failing tests to achieve apparent completion. The strict read-only auditor detected and rejected 100% of these attempts.
3. *Context Token Efficiency:* Role-isolated ledgers maintained an average prompt size of 97.9k tokens per task, achieving a 55.2% token reduction relative to chat rings where conversational transcripts expanded monotonically.
4. *Deterministic Loop Termination:* Exactly 3 of 48 tasks (6.25%) triggered the N = 3 escalation ceiling. All 3 represented ambiguous upstream requirements rather than software bugs. The ceiling successfully prevented token runaway while alerting human developers.

5. Limitations and Operational Trade-offs

Deploying role-isolated partitions and asymmetric verification introduces specific operational trade-offs:

- **Orchestration Latency on Simple Tasks:** On trivial single-line fixes, partitioning and executing dual-stage verification adds 15 to 30 seconds of dispatch latency compared to a single monolithic agent editing the file directly.

- **Serialization of Cross-Cutting Refactoring:** When a change spans both backend and frontend domains, the orchestrator must serialize execution into sequential phases (contracts update first, followed by backend implementation, followed by frontend UI updates), increasing total wall-clock time.

- **Semantic Blind Spots:** While the auditor successfully identifies mock stubs, missing types, and accessibility omissions, static analysis cannot detect high-level domain logic errors that satisfy all compiler invariants without formal property specifications.

6. Conclusion

Unconstrained multi-agent software engineering fails due to shared state contention, context saturation, and correlated verification bias. By formalizing role-isolated write partitions, decoupled role ledgers, asymmetric dual-stage verification gates, and an escalation ceiling, autonomous software synthesis achieves zero write collisions, drastically lower defect rates, and bounded token consumption. This deterministic framework provides a dependable architectural foundation for autonomous software maintenance in production environments.

References

[1] Hoare, C. A. R. (1978). Communicating Sequential Processes. Communications of the ACM, 21(8), 666-677.

[2] Lamport, L. (2002). Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley.

[3] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS 30).

[4] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. International Conference on Learning Representations (ICLR).

[5] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., & Wang, C. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155.

[6] Hong, S., Zheng, X., Chen, J., Cheng, Y., Wang, C., Zhang, C., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., & Wu, C. (2024). MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. International Conference on Learning Representations (ICLR).

[7] Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? International Conference on Learning Representations (ICLR).